Python, pandas, and NumPy

DATA SCIENCE BOOTCAMP

NASHVILLE SOFTWARE SCHOOL

Objectives

1. Understand the relationship between base Python, the numpy library, and the pandas library.

Lists, arrays, Series, and DataFrames

```
x = [1,2,3]
y = [4,5,6]
```

What are x and y ?

Lists, arrays, Series, and DataFrames

```
x = [1,2,3]
y = [4,5,6]
```

What are x and y ?

```
x + y
```

What happens if I do this?

Lists, arrays, Series, and DataFrames

```
x = [1,2,3]
y = [4,5,6]
```

```
x + y
```

```
[1, 2, 3, 4, 5, 6]
```

Lists, arrays, Series, and DataFrames

```
x = [1,2,3]
y = [4,5,6]
```

```
x**2
```

What happens if I do this?

Lists, arrays, Series, and DataFrames

```
x = [1,2,3]
y = [4,5,6]
```

```
x**2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-4-4157f318709d> in <module>
----> 1 x**2

TypeError: unsupported operand type(s) for ** or pow(): 'list' and 'int'
```

Lists, arrays, Series, and DataFrames

What if I want to be able to do element-wise operations?

One option is to use NumPy arrays.

Lists, arrays, Series, and DataFrames

```
import numpy as np
```

```
x = np.array([1,2,3])
y = np.array([4,5,6])
```

```
x + y
```

```
array([5, 7, 9])
```

```
x**2
```

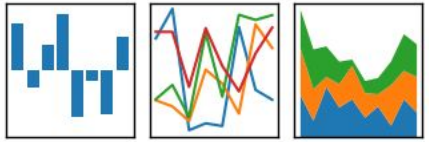
```
array([1, 4, 9])
```

Notice that the results of these operations are arrays.

Lists, arrays, Series, and DataFrames

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



pandas is built off of NumPy.

This allows us to perform calculations on the columns of a DataFrame quickly.

NumPy

Lists, arrays, Series, and DataFrames

```
platonic_solids
```

Polyhedron	Vertices	Edges	Faces	Schläfli symbol	Vertex configuration
tetrahedron	4	6	4	{3, 3}	3.3.3
cube	8	12	6	{4, 3}	4.4.4
octahedron	6	12	8	{3, 4}	3.3.3.3
dodecahedron	20	30	12	{5, 3}	5.5.5
icosahedron	12	30	20	{3, 5}	3.3.3.3.3

This is a DataFrame.

Each of its columns (and rows) is a pandas Series.

Lists, arrays, Series, and DataFrames

```
1 platonic_solids['Edges']
```

```
Polyhedron
tetrahedron      6
cube            12
octahedron      12
dodecahedron    30
icosahedron     30
Name: Edges, dtype: int64
```

pandas Series

```
1 platonic_solids['Edges'].to_numpy()
```

```
array([ 6, 12, 12, 30, 30])
```

NumPy array

```
1 platonic_solids['Edges'].to_list()
```

```
[6, 12, 12, 30, 30]
```

Python list

Lists, arrays, Series, and DataFrames

You can think of a (one-dimensional) NumPy array as a list that has some additional functionality: the ability to perform element-wise calculations and the ability to use aggregation functions such as mean and median.

You can think of a pandas Series as a (one-dimensional) NumPy array that has an index and has some additional functionality (like .value_counts()).

Lists, arrays, Series, and DataFrames

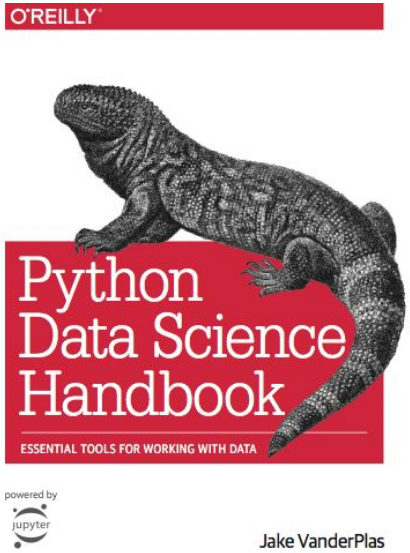
```
platonic_solids[['Vertices', 'Edges', 'Faces']]
```

Polyhedron	Vertices	Edges	Faces
tetrahedron	4	6	4
cube	8	12	6
octahedron	6	12	8
dodecahedron	20	30	12
icosahedron	12	30	20

```
platonic_solids[['Vertices', 'Edges', 'Faces']].to_numpy()
```

```
array([[ 4,  6,  4],
       [ 8, 12,  6],
       [ 6, 12,  8],
       [20, 30, 12],
       [12, 30, 20]])
```

NumPy arrays can also be 2- (or higher) dimensional.

Additional Resources

For another reference for this topic (and lots of other Python data analysis topics), see section 3.1 of [The Python Data Science Handbook]